MODULE 06 · DOMAIN 3 - CORE TERRAFORM WORKFLOW

Reading a Plan. fmt, validate, show, output, console

The symbols, the summary, and the five commands that inspect rather than change.

DOMAIN 3 of 8 COURSE WEIGHT 16% MODULE 6 of 27

admin@sthithapragna.com

The plan is the artefact you approve. If you cannot read it precisely, **approval means nothing**.

The plan symbols

SYMBOL	ACTION
+	Create. This resource does not currently exist
-	Destroy. Terraform will remove this resource
~	Update in place. Changed without destroying and recreating
-/+	Replace. Destroyed, then recreated
<=	Read. A data source will be read during apply, not at plan time

ON THE EXAM

The gap between ~ and -/+ is the difference between a config change and an outage. Read that symbol before anything else.

Reading a single resource block

- A header line names the resource and the action, such as **will be updated in-place** or **must be replaced**.
- **forces replacement** appears beside the specific attribute causing it. That attribute is the whole story.
- **(known after apply)** means the value cannot be computed yet, because it comes from the provider.
- A destroy may be annotated **because it is not in configuration**, meaning you deleted the block.
- Changed values are shown as **old arrow new**, so you can see exactly what moved.

> ON THE EXAM
>
> When a plan surprises you, find the line marked 'forces replacement'. It names the single attribute responsible.

The summary, and what Terraform tells you unprompted

- Every plan ends with a count: **so many to add, to change, and to destroy**.
- If nothing differs, Terraform reports that **no actions need to be taken**. That is a successful plan.
- Terraform separately reports **changes made outside of Terraform** that it noticed during the refresh.
- That section is **drift**. It is the difference between state and reality, not between state and configuration.
- Reading the count first, then the drift note, then the blocks, is a habit worth forming.

> ON THE EXAM
>
> A plan with zero changes and a long drift section is not a quiet plan. Somebody has been editing your infrastructure.

terraform fmt

- **Rewrites configuration files** to the canonical format and style. A subset of the documented style conventions.
- **Intentionally opinionated, with no customisation options.** The goal is consistency between codebases, not your favourite style.
- **By default it only processes the current directory.** Subdirectories need -recursive.
- Terraform's own generated configuration conforms to it, so following it keeps everything consistent.
- Works well as a **pre-commit hook**, which is where HashiCorp suggests putting it.

> ON THE EXAM
>
> It is a formatter, not a linter. It never tells you your configuration is wrong, only that it is untidy.

fmt options

OPTION	EFFECT
`-check`	Do not write. Exit non-zero if any file needs formatting
`-diff`	Show what would change, as a diff
`-recursive`	Process subdirectories too. Off by default
`-list=false`	Suppress the list of filenames that were or would be changed
`-write=false`	Do not overwrite files. Implied by -check

ON THE EXAM

-check plus -recursive is the pipeline combination. It fails the build rather than quietly rewriting the repository.

The canonical format is allowed to change

- **The canonical format may change in minor ways between Terraform versions.**
- HashiCorp does **not** consider new formatting rules to be a breaking change.
- So after upgrading Terraform, **run fmt across your modules** as part of adopting the new version.
- New rules are usually expansions of style already documented, so surprises are small.
- This is why a fmt check can start failing on a repository nobody has touched.

> ON THE EXAM
>
> A formatting check that fails after a CLI upgrade is expected behaviour, not a broken pipeline.

terraform validate

- Checks that a configuration is **syntactically valid and internally consistent**.
- That includes **attribute names and value types** for resources and modules, and references that do not resolve.
- **It does not access remote services.** No provider APIs, no remote state, no credentials.
- It works **regardless of any provided variables or existing state**, so no values are needed.
- Primarily useful for **reusable modules**, and safe to run automatically on save or in CI.

> ON THE EXAM
>
> Validate answers 'is this configuration coherent?'. It never answers 'will this work against my cloud account?'.

validate or plan?

VALIDATE ANSWERS

Is this configuration coherent?

- Structure, types, names, references
- No credentials needed
- No state needed, no variables needed
- Fast, offline, safe on every save

PLAN ANSWERS

What would happen if I applied this?

- Real state, real provider APIs
- Credentials required
- Variable values required
- Slower, and the only real answer

THE TIEBREAKER

A green validate means the code is shaped correctly. It says nothing about whether the change is safe or will succeed.

Two details about validate

- **Validate requires an initialised working directory**, with providers and modules installed. It needs their schemas.
- For CI, initialise without touching the backend: **terraform init -backend=false**, then validate.
- **plan and apply validate automatically** before doing any other work.
- So validate is not a required step in the core workflow. It is a fast, cheap check placed earlier.
- **-json** gives machine-readable output for a pipeline to parse.

> ON THE EXAM
>
> Validate needs init but not a backend. That combination is exactly what a module's CI pipeline wants.

terraform show

- With no arguments, prints the **current state** in human-readable form.
- Given a **saved plan file**, prints that plan in the same readable form.
- That second form is how a reviewer inspects a plan generated elsewhere, such as by a pipeline.
- **-json** produces machine-readable output, and is the supported way to build tooling.
- Reading the raw state file yourself is **not** supported. The format is internal and may change.

> ON THE EXAM
>
> show is the supported interface to state and to plan files. It is the answer whenever a question involves inspecting either.

terraform output

- Reads **root module output values** from state. Not from configuration, and not from the provider.
- With no name it prints all of them. With a name it prints one.
- By default a string is printed **with quotes**, which breaks shell scripts that use the value directly.
- **-raw** prints the bare value with no quotes and no formatting. Use it when piping into something else.
- **Outputs marked sensitive are redacted** in normal output. -raw and -json will still reveal them.

> ON THE EXAM
>
> Outputs come from state, so a value cannot appear until it has been applied. -raw exists for scripts; -json for tooling.

console, graph, version

- **terraform console** is an interactive prompt for evaluating expressions against your configuration and state.
- It is the fastest way to answer 'what does this function actually return'. Module nine leans on it.
- **terraform graph** emits the dependency graph in DOT format, for rendering with a graph tool.
- **terraform version** prints the CLI version **and the versions of installed providers**.
- It also tells you when a newer Terraform release is available.

> ON THE EXAM
>
> console reads state, so it can evaluate expressions against real values rather than hypothetical ones.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Check code without credentials or state	terraform validate, after init -backend=false
Fail the build if code is untidy	terraform fmt -check -recursive
A reviewer must inspect a plan built by CI	terraform show against the saved plan file
A script needs an output value with no quotes	terraform output -raw
Why is this resource being recreated?	The attribute annotated 'forces replacement'
Somebody changed things in the console	The 'changes outside of Terraform' section

ON THE EXAM

None of these commands change infrastructure. If a scenario needs something changed, none of them is the answer.

A team's pipeline runs terraform validate on every pull request and treats a pass as confirmation that the change is safe to merge. Plans are only generated after merge, during deployment.

A change passes validate, merges, and then fails at deployment because the configuration references a subnet that does not exist in the target account.

Why did validate not catch this?

A Validate was run before terraform init, so it had no provider schemas

B Validate checks configuration only and never contacts provider APIs or remote state

C Validate skips data sources, which is where the subnet lookup happens

D Validate requires variable values, and none were supplied in the pipeline

◆ PAUSE AND DECIDE

Why did validate not catch this?

A **Run before init**

Validate does require an initialised directory, and it would have failed loudly rather than passing. The scenario says it passed.

B **Never contacts remote services**

Correct. Validate verifies structure, types and references using local schemas. Whether a subnet exists in an account is a question only plan can ask.

C **Skips data sources**

Invents an exemption. Validate checks data blocks like any other, for attribute names and types. It just cannot execute the lookup.

D **Needs variable values**

Backwards. HashiCorp states validate works regardless of any provided variables or existing state, which is why it suits reusable modules.

THE SIGNAL

Nothing failed. The pipeline asked a structural question and treated the answer as a contextual one.

admin@sthithapragna.com

> A repository has passed its formatting check for two years. The pipeline runs terraform fmt -check -recursive. Nobody has changed the Terraform code this week.
>
> After the pipeline image was rebuilt and picked up a newer Terraform CLI, the formatting check now fails on forty files across the repository.

What is the most likely explanation?

A The newer CLI applies additional formatting rules. This is expected and not a breaking change

B The -recursive flag behaves differently in newer versions and is now scanning vendor directories

C The repository was reformatted by a third-party tool during the image rebuild

D fmt -check is non-deterministic and should be replaced with fmt -diff

◆ PAUSE AND DECIDE

What is the most likely explanation?

A New rules in a new CLI

Correct. HashiCorp states the canonical format may change in minor ways between versions and that new rules are not a breaking change.

B -recursive changed

Plausible-sounding and unsupported. Recursive has meant subdirectories since it was introduced, and the count would be far larger.

C A third-party tool

The image rebuild changed the tool, not the repository. Nothing in the pipeline writes back to source.

D fmt is non-deterministic

It is entirely deterministic for a given version, which is the whole point of a canonical format. -diff shows more detail; it changes no outcome.

THE SIGNAL

The planted detail is that nobody changed the code. When inputs are constant and output changes, the tool changed.

> A deployment script captures a load balancer hostname from Terraform and passes it to a health check command. The script uses terraform output lb_hostname and stores the result in a shell variable.
>
> Every health check fails. The logs show the tool attempting to reach a hostname that includes double quotation marks at both ends.

What should the script use instead?

A terraform output -raw lb_hostname

B terraform show lb_hostname

C terraform console, piping the output name into it

D terraform output -json lb_hostname, then use the value directly

◆ PAUSE AND DECIDE

What should the script use instead?

A output -raw

Correct. -raw prints the bare value with no quoting or formatting, which is exactly what a shell variable needs.

B show with a name

show prints whole state or a whole plan file. It does not take an output name and cannot select a single value.

C console piping

Console can evaluate the expression, and it is an interactive tool wrapped in a script. -raw exists so you do not have to.

D output -json

Closer, and it returns a JSON document that still needs parsing. It would leave the quotes unless something extracted the value.

THE SIGNAL

Default output is formatted for a human reading a terminal. -raw is formatted for another program.

> An engineer changes one tag on a managed database and expects an in-place update. The plan instead shows the database with -/+ and the header 'must be replaced'. Elsewhere in the block, one line reads 'forces replacement'.
>
> She is under time pressure and the change is considered routine.

What should she do?

A. Apply. The plan symbol indicates an update with a brief interruption

B. Read the attribute marked 'forces replacement', because the database will be destroyed and recreated

C. Re-run plan with -refresh=false, since stale state is the usual cause of a replacement

D. Apply with -replace on the database, which performs the same change more safely

◆ PAUSE AND DECIDE

What should she do?

A. Apply, it is an update

Misreads the symbol. -/+ is destroy then create, not update. For a database that means data loss unless something else protects it.

B. Read the forces replacement line

Correct. That annotation names the single attribute responsible, which is both the explanation and, usually, the thing to change.

C. Plan with -refresh=false

Skips reading reality, so it would hide information rather than add any. Replacement comes from a configuration difference, not from stale data.

D. Apply with -replace

Asks for the destruction explicitly. It is the correct tool for a degraded resource and here it would guarantee the outcome she wants to avoid.

THE SIGNAL

Time pressure plus a routine-looking change is when replacement symbols get skimmed. That combination is the actual hazard.

Module 6 recap

- **Plus create, minus destroy, tilde update in place, minus-slash-plus replace.** The last one is the one to slow down for.
- **forces replacement** names the single attribute responsible. Find it before you argue with the plan.
- **fmt is a formatter, not a linter,** and it is not recursive by default.
- **The canonical format may change between versions.** A check failing after an upgrade is expected.
- **validate never contacts remote services.** Coherent is not the same as correct or permitted.
- **show inspects state and plan files. output -raw is for scripts.** Both read from state.

> ON THE EXAM
>
> Everything in this module is read-only. If a scenario needs infrastructure changed, none of these commands is the answer.